



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Work Assignment 2 –An Approach to Oblivious, Decentralized IAM Integrated with a Searchable, Secure, Encrypted Block-Storage Service

Name: Nicole Arquissandas

Student number: 75026

Introduction

This project implements a secure and oblivious storage system composed of three logical servers:

- **OAS** – Oblivious Authentication Server: registers users and authenticates them without ever receiving their password in clear.
- **OAMS** – Oblivious Authorization & Management Server: manages file-sharing metadata and encrypted file keys.
- **OBSS** – Oblivious Block Storage Server: stores only encrypted file blocks and search tokens for keywords.

The client is responsible for:

- managing all long-term and per-file cryptographic keys,
- encrypting/decrypting file contents,
- generating oblivious keyword tokens,
- interacting with OAS, OAMS, and OBSS according to the project protocols.

The main goals are:

- Ensure that no single server alone can reconstruct file contents or learn keywords.
- Preserve oblivious search: the storage server never sees keywords in clear, only tokens.
- Support secure file sharing with permissions (GET, SEARCH, GET+SEARCH) using encrypted keys.
- Show that the system is robust against honest-but-curious administrators and passive man-in-the-middle attackers.

Overall System Architecture

The system follows a client/server architecture with three independent services:

Client:

- Classes: CLTest, ClientCore, lamClient, crypto helpers.

OAServer (OAS) – listens on port 6000.

OAMSServer (OAMS) – listens on port 7000.

OBSServer (OBSS) – reuses the server from Project 1 (port 5000).

In the project setup they run on the same physical machine for simplicity, but the protocols assume they could be on different machines.

Responsibilities of each component

Client (CLTest + ClientCore + lamClient)

- Generates and stores each user's ECC keypair:
 - user_ec_priv.b64, user_ec_pub.b64.
- Generates and stores global symmetric keys (Project 1) and derived per-file keys.
- Implements all commands:
 - Project 1:
 - PUT, LIST, SEARCH, GET, GET CHECKINTEGRITY
 - Project 2:
 - REG, LOGIN, REGMOD, REGDEL
 - SHOWPUB
 - SHARE, SHAREDEL, LISTSHARED
 - GETSHARED, SEARCHSHARED
- Maintains local indexes:
 - fileIndex: filename -> list<blockId>
 - fileKeywords: filename -> list<keywords>
 - fileIds: filename -> fileId (UUID without dashes)

OAS

- Maintains user registrations:
 - userPubKeyB64 -> Registration(passwordRecord, attributesHash)
- Holds an ECC keypair of its own:
 - oas_ec_priv.b64, oas_ec_pub.b64
- Uses its ECC keypair to:
 - sign challenges and authentication responses,
 - sign authentication tokens.
- Exposes commands:
 - CREATE_REG – create registration
 - MODIFY_REG – modify registration (change password record)

- DELETE_REG – delete registration
- AUTH_STEP1, AUTH_STEP2 – two-step password-based authentication

OAMS

- Maintains sharing metadata:
 - (ownerPubKeyB64, fileId, authorizedPubKeyB64) -> { permissions, encFileKey, encKwKey }
- For each share, stores file keys encrypted under the authorized user's public key.
- Exposes commands:
 - SHARE_CREATE
 - SHARE_DELETE
 - SHARE_GETKEYS
 - SHARE_LIST_MY

OBSS

- Stateless regarding users and permissions.
- Maintains:
 - blockId -> encryptedBlob
 - keywordToken -> set<fileId>
- Exposes commands:
 - STORE_BLOCK(blockId, blob, [tokens])
 - GET_BLOCK(blockId)
 - SEARCH(token) -> list<fileId>

OBSS never sees usernames, passwords, or encryption keys. It only sees opaque IDs and ciphertext.

Cryptographic Design and Key Management

Supported algorithms

As stated in project1 report the encryption algorithm for file blocks is configurable via cryptoconfig.txt. Two modes are supported:

- AES_GCM

- CHACHA20_POLY1305
- AES_CBC_HMAC – AES-CBC for confidentiality + HMAC-SHA256 for integrity.

A CryptoSuite is created via CryptoFactory, which supports:

- a data key (dataKey) for encryption,
- an optional MAC key (macKey) for AES_CBC_HMAC,
- Additional Authenticated Data (AAD) to bind blocks to their fileId and block index.

Key hierarchy

On the client, there is a global ClientCore.KEYS structure that contains long-term symmetric secrets:

- Global data key (used as a master key) dataKey – used to derive per-file data keys.
- Global keyword key kwKey – used to generate keyword tokens.
- Global MAC key macKey – used only in AES_CBC_HMAC mode.

These global keys are generated once on the client and stored locally in the client directory. They are never sent to any server.

Per-file keys:

To avoid storing a separate random key for every file, the client derives per-file encryption keys in a deterministic way from:

- the global master key dataKey and
- the fileId (a UUID string without dashes, unique per file).

In the current implementation:

- **Block encryption** uses a **per-file data key** fk.dataKey, derived as above from dataKey and fileId using a deterministic HMAC-based KDF.
- **Keyword tokens** still use the **global** kwKey (ClientCore.KEYS.kwKey) instead of a per-file keyword key. This keeps SEARCH easier and consistent with Project 1.
- For **sharing**, the client encrypts and sends to OAMS:
 - the per-file encryption key (fk.dataKey), and
 - the keyword key (here, the global kwKey), both encrypted under the authorized user's ECC public key.

Keyword tokens

For each keyword kw:

1. The client normalizes (lowercases, trims) the keyword.
2. It generates a token with HMAC:

3. String tok = token(KEYS.kwKey, kw);

Conceptually:

token = HMAC-SHA256(kwKey, kwBytes)

The OBSS receives only this token, it never sees the keyword in plaintext and stores:

- token -> set<fileId> where that keyword was associated.

OBSS can support SEARCH but is oblivious to the actual keywords.

User keys (ECC)

Each client generates an ECC keypair on first use:

- Private key: user_ec_priv.b64
- Public key: user_ec_pub.b64 (this also acts as the user ID)

These keys are used for:

- Signing requests to OAS:
 - registration, authentication, modification, deletion.
- Signing requests to OAMS:
 - share creation and deletion.
- Decrypting file keys shared by other users (via OAMS).

The OAS has its own ECC keypair:

- oas_ec_priv.b64, oas_ec_pub.b64

The OAS uses its private key to:

- sign authentication challenges and responses,
- sign authentication tokens.

Protocol Flows

User lifecycle in OAS

- Registration

Client command:

```
java CLTest REG <password>
```

Flow:

1. The client generates (or loads) its ECC keypair.
2. lamClient.registerAtOAS(password):

- Uses PasswordUtil.deriveFromPassword(...) to derive a PasswordRecord (PBKDF2 with salt and iterations).
- Uses "0" as a dummy attributes hash (no extra attributes in this project).
- Signs the string:
- "CREATE_REG|userPubKeyB64|passwordRecord|attributesHash"
- Sends to OAS:
 - CREATE_REG
 - userPubKeyB64
 - passwordRecord
 - attributesHash
 - signature

3. OAS:

- Reconstructs userPk from userPubKeyB64.
- Verifies the signature with the user's public key.
- Checks if a registration already exists. If not, stores:
 - Registration(userPubKeyB64, passwordRecord, attributesHash).
- Replies with:
 - OK / NOK and a message.

Notes:

- The client proves ownership of the ECC private key at registration
- no password in clear is sent; only a PBKDF2 record.
- **Authentication (LOGIN → AUTH_STEP1 + AUTH_STEP2)**

User command:

```
java CLTest LOGIN <password>
```

Step 1 – AUTH_STEP1

1. The client sends:

- AUTH_STEP1
- userPubKeyB64
- signature of "AUTH_STEP1|userPubKeyB64" with the user's private key.

2. OAS:

- Verifies the signature, proving possession of the private key.
- Looks up the Registration for userPubKeyB64.
- If not found: returns NOK.
- Otherwise:
 - Generates a 32-byte random challenge.
 - Stores it in pendingChallenges[userPubKeyB64].
 - Returns:
 - AUTH_CHALLENGE
 - challengeB64
 - passwordRecord
 - OAS signature over:
 - "AUTH_CHALLENGE|userPubKeyB64|challengeB64|passwordRecord"

Step 2 – AUTH_STEP2

3. On the client:

- It decodes passwordRecord.
- Uses PasswordUtil.deriveKeyForAuth(password, record) to derive a key from the user-typed password.
- Decodes the challenge from Base64.
- Computes:
 - response = HMAC-SHA256(derivedKey, challenge)
- Base64-encodes response and sends:
 - AUTH_STEP2
 - userPubKeyB64
 - challengeB64
 - responseB64
 - signature over:
 - "AUTH_STEP2|userPubKeyB64|challengeB64|responseB64"

4. OAS:

- Verifies the ECC signature.
- Loads the Registration and retrieves the stored challenge.
- Recomputes:
 - $\text{expected} = \text{HMAC-SHA256}(\text{storedKeyFromPasswordRecord}, \text{challenge})$
- Compares expected with clientResponse. If equal:
 - Calls `TokenUtils.createToken(oas_priv, userPubKeyB64, validityMillis)`.
 - Sends:
 - AUTH_OK
 - token
 - OAS signature over "AUTH_OK|userPubKeyB64|token".
- Otherwise, returns NOK.

5. The client prints Authenticated and saves the token in `auth_token.txt` for later use by `IamClient`.

Notes:

- The password never leaves the client.
- The OAS only sees HMAC values and PBKDF2 records.
- Replay attacks are prevented because the challenge is fresh and stored per user.
- Man-in-the-middle cannot recover passwords or plaintext from the protocol messages.

- **Modify registration**

User command:

```
java CLTest REGMOD <oldPassword> <newPassword>
```

Flow (current implementation):

1. `IamClient.modifyRegistration(oldPassword, newPassword)`:
 - Locally derives a new `PasswordRecord` from `newPassword`.
 - Prepares
"MODIFY_REG|userPubKeyB64|newPasswordRecord|attributesHash".

- Signs that string with the user's ECC private key.
- Sends to OAS:
 - MODIFY_REG
 - userPubKeyB64
 - newPasswordRecord
 - attributesHash
 - signature

2. OAS:

- Reconstructs the user's public key.
- Verifies the signature.
- If a registration exists for userPubKeyB64, it updates:
 - passwordRecord = newPasswordRecord
 - attributesHash = newAttrsHash.
- Responds with OK / NOK and a message.

Notes:

- The change of password record is authorized only if the user proves possession of the ECC private key.
 - The protocol does not send any password in clear; only the new PBKDF2 record.
- **Delete registration**

User command:

```
java CLTest REGDEL <password>
```

Flow (simplified):

1. `IamClient.deleteRegistration(...)`:
 - Builds "DELETE_REG|userPubKeyB64".
 - Signs with the ECC private key.
 - Sends:
 - DELETE_REG
 - userPubKeyB64

- signature
2. OAS:
 - Verifies the signature.
 - Removes the registration entry for userPubKeyB64.
 - Replies OK / NOK with a message.
 3. On success, the client clears the local authToken and deletes auth_token.txt.

Again, authorization is based on the ECC keypair identity rather than password.

Project 1 operations with OBSS

PUT

User:

```
java CLTest PUT <path/dir/file> "<keywords>"
```

Flow:

1. The client generates a fileId (UUID without dashes) and stores:
 - fileIds[filename] = fileId.
2. Derives per-file keys: FileKeys fk = deriveFileKeys(fileId).
3. Loads cryptoconfig.txt and builds a matching CryptoSuite.
4. Reads the input file in chunks (BLOCK_SIZE bytes).
5. For each block index idx:
 - Computes:
 - blockId = fileId + "_block_" + idx
 - aad = (fileId + ":" + idx).getBytes()
 - Encrypts:
 - blob = suite.encrypt(plaintext, aad);
 - Sends to OBSS:
 - STORE_BLOCK
 - blockId
 - blob.length

- blob
- For idx == 0 only:
 - Generates keyword tokens for each non-blank keyword.
 - Sends the number of tokens and each token as UTF.
- For other blocks:
 - Sends a token count of 0.

6. OBSS replies "OK" per block, and the client stores block IDs in fileIndex[filename].

LIST

Simple local command:

```
java CLTest LIST
```

It prints all filenames in fileIndex (local mapping).

SEARCH (Project 1 keyword search)

User:

```
java CLTest SEARCH <keyword>
```

Flow:

1. The client generates a token using the global kwKey.
2. Sends to OBSS:
 - SEARCH
 - token
3. OBSS returns an integer count and then count fileIds.
4. The client maps each fileId back to a filename with fileIds, and prints the list.

OBSS learns only the token (HMAC result), not the underlying keyword.

GET by filename

User:

```
java CLTest GET <filename> <outputDir>
```

Flow:

1. Client finds fileId and block list from fileIds and fileIndex.
2. Derives per-file FileKeys again.

3. Rebuilds CryptoSuite with the per-file dataKey.
4. For each block in order:
 - Sends GET_BLOCK blockId to OBSS.
 - Receives blob and decrypts with suite.decrypt(blob, aad) using the same AAD.
 - Writes plaintext to the output file.

CHECKINTEGRITY

User:

```
java CLTest GET CHECKINTEGRITY <path/dir/file>
```

Implementation:

1. Extracts filename from the path and uses fileIndex[filename] to get block IDs.
2. For each block index i:
 - Sends GET_BLOCK blockId.
 - Receives the encrypted blob.
 - Attempts to decrypt using SUITE.decrypt(blob, aad) in a try/catch.
 - If any decryption fails (AEAD tag or HMAC mismatch), prints:
 - Integrity FAIL on block i
 - and aborts.
3. If all blocks decrypt correctly, it prints:
4. Blocks OK (<cipher name>).
5. It then verifies keyword linkage:
 - For each keyword associated with the file:
 - Computes a token with kwKey.
 - Sends SEARCH token to OBSS.
 - Checks whether the fileId appears in the returned list.
 - If any keyword is missing, it aborts.
6. If everything passes:
7. Keywords OK (<number_of_keywords>).
8. CHECKINTEGRITY: PASS for <filename>

Sharing with OAMS

Creating a share

Example from Alice:

```
java CLTest LOGIN alicePw  
java CLTest SHARE secret_alice.txt <BOB_PUBKEY> GET+SEARCH
```

Flow:

1. CLTest uses `core.getFileIdForName(filename)` to resolve `fileId`.
2. Uses `core.getFileKeysForFileId(fileId)` to obtain `FileKeys fk` (per-file data key).
3. Calls:
4. `iam.shareFile(fileId, authorizedPubKeyB64, permissions, fk.dataKey, ClientCore.KEYS.kwKey);`
5. `IamClient.shareFile`:
 - Builds the authorized user's public key from `authorizedPubKeyB64`.
 - Encrypts:
 - `encFileKey = EccUtils.encryptFor(authzPk, fileDataKey)`
 - `encKwKey = EccUtils.encryptFor(authzPk, fileKwKey)`
(this is the global `kwKey` in the current design)
 - Prepares a string to sign:
 - `"SHARE_CREATE|authToken|fileId|authorizedPubKeyB64|permissions|encFileKey|encKwKey"`
 - Signs it with the owner's ECC private key.
 - Sends to OAMS:
 - `SHARE_CREATE`
 - `authToken`
 - `fileId`
 - `authorizedPubKeyB64`
 - `permissions`
 - `encFileKey`
 - `encKwKey`

- signature.

6. OAMS:

- Validates authToken using the OAS public key.
- Verifies the ECC signature of the owner.
- Stores (ownerPk, fileId, authorizedPk) -> {permissions, encFileKey, encKwKey}.
- Responds with "OK" and a status message.

Listing shares received

On the authorized user (ex. Bob):

```
java CLTest LOGIN bobPw  
java CLTest LISTSHARED
```

Flow:

1. lamClient.listSharesForMe():
 - Sends SHARE_LIST_MY, authToken, and a signature.
2. OAMS returns:
 - "OK", then n, and for each entry:
 - fileId
 - permissions
 - encKwKey
3. The client:
 - Decrypts encKwKey with its ECC private key.
 - Builds a SharedForMe(fileId, permissions, kwKey) object.

CLTest LISTSHARED then prints:

```
Files shared with me:  
- fileId=<...> perms=GET+SEARCH  
- fileId=<...> perms=GET
```

Searching in shared files

Example:

```
java CLTest SEARCHSHARED project2
```

Flow:

1. CLTest calls iam.listSharesForMe().
2. For each SharedForMe sf where sf.permissions contains "SEARCH":
 - Calls core.searchSharedInFile(keyword, sf.fileId, sf.kwKey).
3. searchSharedInFile:
 - Uses sf.kwKey to build a keyword token.
 - Sends SEARCH token to OBSS.
 - OBSS returns all fileId that contain that token.
 - The client checks whether sf.fileId is in that list.
 - Returns true if yes.
4. CLTest prints:

Shared files containing "project2":
- <fileId_of_secret_alice.txt>

Getting a shared file

Example:

```
java CLTest GETSHARED <fileId> out/bob_secret.txt
```

Flow:

1. CLTest calls iamClient.getSharedKeys(fileId):
 - Sends SHARE_GETKEYS, authToken, fileId, and a signature.
 - OAMS checks token + signature and returns:
 - "OK"
 - permissions
 - encFileKey
 - encKwKey
 - The client decrypts:
 - fileKey = EccUtils.decryptFrom(userSk, encFileKey);
 - kwKey = EccUtils.decryptFrom(userSk, encKwKey);
 - Returns a SharedKeys object.

2. CLTest enforces permissions:
 - If permissions does not contain "GET", it aborts with a message.
3. If allowed, CLTest calls:
4. `core.getSharedFile(fileId, outputPath, fileKey);`
5. `getSharedFile`:
 - Builds a CryptoSuite with the provided fileKey.
 - Retrieves all blocks from OBSS for that fileId (`fileId_block_0`, `fileId_block_1`, ...).
 - Decrypts them using the same AAD scheme and writes to outputPath.

The authorized user can reconstruct the file content, but only because they received the file's encryption key via OAMS.

Deleting a share

Example:

```
java CLTest SHAREDEL secret_alice.txt <BOB_PUBKEY>
```

Flow:

1. CLTest resolves filename -> fileId.
2. Calls `IamClient.deleteShare(fileId, authorizedPubKeyB64)`:
 - Signs "SHARE_DELETE|authToken|fileId|authorizedPubKeyB64".
 - Sends SHARE_DELETE, authToken, fileId, authorizedPubKeyB64, signature to OAMS.
3. OAMS:
 - Validates the token and signature.
 - Removes the corresponding sharing entry if it exists.
 - Replies with "OK / NOK" and a message.

After this, the authorized user:

- No longer sees the file in LISTSHARED.
- GETSHARED fileId fails with "No shared keys or no permission."

Conclusion

The implemented system combines:

- Secure, password-based authentication with OAS using PBKDF2 records and HMAC over fresh challenges.
- Oblivious, encrypted storage of file blocks and keywords in OBSS, where the server never sees plaintext or keywords.
- Controlled file sharing through OAMS, where file encryption keys and keyword keys are encrypted under the authorized user's ECC public key and associated with fine-grained permissions.

The client is the only component that sees the full picture:

- user passwords,
- ECC private keys,
- file contents,
- per-file data keys,
- keyword keys.

Each server has access only to the minimum information required to perform its role, and the architecture is robust against honest-but-curious administrators and passive network observers.

Server separation:

- OAS only sees:
 - public keys,
 - password records,
 - signatures,
 - and issues tokens.
- OAMS only sees:
 - anonymized fileId,
 - encrypted file keys (encFileKey, encKwKey),
 - permissions (GET, SEARCH, GET+SEARCH),
 - but never plaintext content or filenames.
- OBSS only sees:
 - encrypted blocks,
 - HMAC-based keyword tokens,
 - block IDs and file IDs (opaque identifiers).

Honest-but-curious administrators:

- An admin of OBSS cannot decrypt any file or recover keywords.
- An admin of OAMS cannot decrypt file keys (they are encrypted for the authorized user).
- An admin of OAS cannot see passwords, only password records.

Man-in-the-middle (passive):

- All traffic uses either:
 - encrypted blocks,
 - PBKDF2 records,
 - HMACs,
 - signatures,
 - or tokens.
- No plaintext passwords or file contents are transmitted.

Demonstration

Tests were performed with separate client directories (client-alice, client-bob, client-carol) to simulate different users, each with its own ECC keys, local indexes, and files.

On Alice's terminal:

Im testing all functionalities of project 1, but more importantly im putting 2 files "secret_alice.txt" with keywords project2 and secret and "notes_alice.txt" with keywords project2 and notes on OBSS and registering on OAS.

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest REG alicePw
REGISTER: OK - Registered
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest PUT .\clientfiles\secret_alice.txt "project2 secret"
.
PUT: stored secret_alice.txt (1 blocks)
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest PUT .\clientfiles\notes_alice.txt "project2 notes"
.
PUT: stored notes_alice.txt (1 blocks)
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LIST
LIST:
- notes_alice.txt
- secret_alice.txt
```

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest SEARCH notes
SEARCH results:
- notes_alice.txt
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest GET secret_alice.txt out/
.
GET: reconstructed to C:\Users\nicol\IdeaProjects\Project_SSC\client-alice\out\secret_alice.txt
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest GET CHECKINTEGRITY clientfiles/secret_alice.txt
Blocks OK (AesGcm).
Keywords OK (2).
CHECKINTEGRITY: PASS for secret_alice.txt
```

On Bob's terminal:

I register bob on OAS and ask for his publickey

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest REG bobPw
REGISTER: OK - Registered
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java SHOWPUB
Error: Could not find or load main class SHOWPUB
Caused by: java.lang.ClassNotFoundException: SHOWPUB
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest SHOWPUB
My public key (user ID) is:
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEcNX3VjYZSEPe08sRzMewrRHaLbNOGNrDdhKVL08RlygoK/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw==
```

On Carol's terminal:

I register bob on OAS and show ask for her publickey

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest REG carolPw
REGISTER: OK - Registered
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest SHOWPUB
My public key (user ID) is:
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAEmpX+9h2yPR340GeSU1zUgsNj87gyXw8Pkzw9VCTaUjU+ZTWclQEgAUEPTXULHp0Ub8kMbDwfjtmInpe15vcgfQ==
```

Then on Alice's terminal:

I share permissions of file alice_secret.txt to bob with to search with the tokens and get the file and share permissions of file notes_alice.txt to carol to only get the file, but before that alice must login to be able to do that

Login in:

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest SHARE secret_alice.txt MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEcNX3VjVZSEPe08sRzMewrRHaLbNOGNrDdhKVL08RlygoK
/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw== GET+SEARCH
You must LOGIN first.
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LOGIN aliceP
AUTH_STEP2 failed: Invalid credentials
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LOGIN alicePw
Authenticated:
Token = TUZrd0V3WUhlb1pJemowQ0FRWUllb1pJemowREFRY0RRZ0FFK0dEdmLYYmdRWHBCRDJ6N3lr0EtIS6F4U09NVW1nNEErVUtTUTJyRDI5WTc4YWlNanhMTDlLbm5qdExHa2U3NmVRMzBhQlU0czc1anRwWlM0Ni8rMkE9PXMxNzY1
NDE3NTE2ODcxZDZj0MTY4NzE=.MEQICBaznwZDE6ipn7HkQ605B0o4eaP5HCosz2EAX01oQMqAiAVvB9THStuVf9r8yithcNd6Pd/61mCnxW8c0VSXqG37Q==
My public key (user ID) is:
MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAE+GDviXbgQXpBD2z7yk8KHaxS0MUmg4A+UKSQ2rD29Y78aiMjhLL9KnnjtLGke76eQ30aBU4s75jtpZS46/+2A==
```

Sharing:

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest SHARE secret_alice.txt MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEcNX3VjVZSEPe08sRzMewrRHaLbNOGNrDdhKVL08RlygoK
/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw== GET+SEARCH
SHARE_CREATE: OK - Share created
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest SHARE notes_alice.txt MFkwEwYHkoZiZj0CAQYIKoZiZj0DAQcDQgAEmpX+9h2yPR340GeSU1zUgsNj87gyXw8Pkzw9VCTaUjU+ZT
Wc1QEgAUEPTXULHp0Ub8kMbDwfjtmInpe15vcgfQ== GET
SHARE_CREATE: OK - Share created
```

Then on Bob's terminal:

I check for the file alice shared with be seeing if bob has permission to get the file as well as search by the files keywords(that alice specified when putting the file in OBBS)

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest SEARCHSHARED secret
You must LOGIN first.
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest LOGIN bobPw
Authenticated:
Token = TUZrd0V3WUhLb1pJemowQ0FRWULb1pJemowREFRY0RRZ0FFY0SVM1ZqWVpTRVBLTzhzUnpNZdyUkhTGJ0T0d0ckRkaEtWTG84UmX5Z29LL0FEWL8mYUtUZFNzeW1LRUduMDYxZDUZ2Z5NM0VzUVpVdLEwY0x0Wnc9PKwxNzY1
NDE3NjI2MTIxY0MTg1MjYxMjE=.MEUCIAfk5ExeCzAiVumadTnngH4LKtKRXsyUoraqKCf07nDhAIEA2PL8deP+6Los+c0c671gKi89zFw0R4EjIMW4zCIBViU=
My public key (user ID) is:
MFkwEwYHKOZiZj0CAQYIKoZiZj0DAQcDQgAECNX3VjYZSEPe08sRzMEwrRHaLbNOGNrDdhKVL08RlygoK/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw==
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest SEARCHSHARED secret
Shared files containing "secret":
- fa772bbcfbb54645a8472315727e076c
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest GETSHARED fa772bbcfbb54645a8472315727e076c alice_secret_shared.txt
Shared file written to alice_secret_shared.txt
```

On Carol's terminal:

I check for the file alice shared with be seeing if bob has permission to get the file but not search by the files keywords

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest LOGIN carolPw
Authenticated:
Token = TUZrd0V3WUhLb1pJemowQ0FRWULb1pJemowREFRY0RRZ0FFbV84KzloMnLQjM0T0dLU1UxeLVnc05q0DdneVh30FBrenc5VknUYVvqVStaVfDjbFFF20FVRVBWUFVMSHAWVWI4a01iRHdmanRtSWSw2TE1dmNnZLE9PXN
NDE3NzIxMzIzY0MTg1MjYxMjE=.MEYCIQDhktKzGf020L7/pNOWu1xLmgWPxz37QSHZTR2z6IK0wIhALAqmdcLmYsuuQ0S7+9tKZL6kRN+AhAN+mqICEN+9rND
My public key (user ID) is:
MFkwEwYHKOZiZj0CAQYIKoZiZj0DAQcDQgAECNX3VjYZSEPe08sRzMEwrRHaLbNOGNrDdhKVL08RlygoK/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw==
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest SEARCHSHARED notes
No shared files contain keyword "notes".
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest LISTSHARED
Files shared with me:
- fileId=b8596993d2a34deeb6fc4d23e3b3992 perms=GET
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol> java CLTest GETSHARED b8596993d2a34deeb6fc4d23e3b3992 alice_notes_shared.txt
Shared file written to alice_notes_shared.txt
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-carol>
```

Then on Alice's terminal:

I delete the permissions I gave to bob to search and get the file alice_secret.txt

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest SHAREDDEL secret_alice.txt MFkwEwYHKOZiZj0CAQYIKoZiZj0DAQcDQgAECNX3VjYZSEPe08sRzMEwrRHaLbNOGNrDdhKVL08Rly
goK/ADZPfakTdSsymeEGn061d53gnM3EsQZUvQ0cLNZw==
SHARE_DELETE: OK - Share deleted
```

Then on Bob's terminal:

I check that the file is no longer shared with bob and he cannot get or search for the file

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest SEARCHSHARED secret
No files shared with you.
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest GETSHARED fa772bbcfbb54645a8472315727e076c alice_secret_shared.txt
SHARE_GETKEYS failed: No sharing entry
No shared keys or no permission.
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-bob> java CLTest LISTSHARED
No files shared with you.
```

Finally on Alice's terminal:

I check that I can modify alices registration (modifying the password) and check that I can delete Alices registration from OAS.

```
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest REGMOD alicePw alicePw2
REGMOD: OK - Modified
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LOGIN alicePw
AUTH_STEP2 failed: Invalid credentials
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LOGIN alicePw2
Authenticated:
Token = TUZrd0V3WUhLb1pJemowQ0FRWUlb1pJemowREFRY0RRZ0FFK0dEdmLYYmdRlWHBCRDJ6N3lr0EtISGF4U09NVW1nNEErVUtTUTJyRDlSWTc4YWlNanhMTDlLbm5qdExHa2U3NmVRMzBhQlU0czc1anRwWlM0Ni8rMkE9PXwxNzY1NDIwNDUyOTk4fDE3NjU0MjEzNTI5OTg=.MEUCIEZdqH0ocZKWyT8FFRuswi7zd8nmw6RFSOL3tfmLjZCuAIEA2ZTH29qjk3xLkeenpzGexikVLqA68tftZvVWChN+BbY=
My public key (user ID) is:
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAE+GDviXbgQXpBD2z7yk8KHaxSOMUmg4A+UKSQ2rD29Y78a1MjhLL9KnnjtLGKe76eQ30aBU4s75jtpZS46/+2A==
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest REGDEL alicepw2
REGDEL: OK - Deleted
(base) PS C:\Users\nicol\IdeaProjects\Project_SSC\client-alice> java CLTest LOGIN alicePw2
AUTH_STEP1 failed: NOK - Invalid credentials
```